Auditoría del módulo de tareas y notificaciones

Fecha: 20 de octubre de 2026. Base Git: `e7bffc6`, incluyendo los cambios locales presentes al revisar. Instrucción aplicada: Agents.md.

1. Resumen ejecutivo

El módulo tiene una implementación operativa amplia: tareas manuales y automáticas, agenda, comentarios, avisos internos, recordatorios por correo, resúmenes Premium y una cola offline con identificación de operaciones y detección de conflictos. **No puede calificarse como completamente íntegro ni seguro todavía.**

Se identificaron **18 hallazgos: 10 P1 y 8 P2**. P1 requiere atención prioritaria por aislamiento, autorización, integridad operativa o suscripción; P2 afecta fiabilidad, trazabilidad offline o experiencia. Las prioridades no representan una explotación observada en producción.

Los problemas principales son:

- Las altas y ediciones de tareas aceptan campos internos y operadores MongoDB. Se puede introducir otra organización/finca o eludir la validación del responsable.
- Las restricciones de tareas sanitarias y protocolos difieren entre rutas. Algunas ediciones permiten marcar la tarea como completada sin ejecutar el procedimiento de origen.
- El reintento idempotente se resuelve antes de comprobar si el usuario conserva acceso a la tarea.
- La tarea se guarda antes de actualizar efectos y bitácora. Un fallo posterior deja cambios parciales y el reintento puede dar éxito sin repararlos.
- Los avisos de tareas manuales reciben un actor incompatible con la normalización de identificadores y fallan silenciosamente para el usuario.
- El reparto de notificaciones por rol y los correos operativos no verifican el acceso individual a las fincas involucradas.
- Las rutas de tareas y notificaciones no exigen suscripción vigente.
- La fecha de ejecución offline se reemplaza por la de recepción; también hay discrepancias entre fechas civiles, vencimientos y zonas horarias.

Controles que sí existen: autenticación y finca activa; filtros de organización/finca en consultas ordinarias de Tarea; selección de usuarios asignables con membresía y acceso a finca; lectura normal de tareas restringida por asignación; notificaciones restringidas al destinatario; índice único para deduplicar avisos; secreto de cron comparado con `timingSafeEqual`; bloqueo compartido del trabajo programado; control de plan para correos operativos y resúmenes; aislamiento local por organización, finca y usuario.

Alcance y comprobaciones

Se inspeccionaron modelos, plugin de aislamiento, autenticación, permisos, planes, rutas y controladores, servicios de tareas/notificaciones/correo, generadores de los módulos conectados, componentes React, API cliente, almacenamiento offline, sincronizador, configuración PWA y pruebas relacionadas.

Validación realizada:

Comprobación	Resultado	Alcance
`cd backend; npm.cmd test`	**254/254 aprobadas**	Suite existente del backend en este árbol de trabajo.

Comprobación	Resultado	Alcance
`cd frontend; npm.cmd run test:lunar`	**6/6 aprobadas**	Fases, iluminación, fechas y clasificación de tareas lunares.
Sondas Node aisladas sobre modelos, servicios y controlador reales	**18/18 confirmaciones**	Persistencia y servicios externos sustituidos; incluye controles positivos y reproducción de defectos.

Las sondas confirmaron: conservación de tenant ajeno en alta; actualización que filtra el tenant actual pero escribe otro; error del actor de notificación y éxito con un ID válido; replay de tarea ajena; bypass sanitario y de responsable con `$set`; finalización directa de bandas y engorde; completar una cancelada por `/estado`; replay después de fallo de bitácora sin reparación; fecha real ignorada y versión inválida tolerada; evento tipado pese a `generaBitacora:false`; rechazo normal de lectura ajena y de finalización por Consulta; reparto sin filtro de finca; deduplicación que conserva un aviso leído; HTML sin escapar; desplazamiento de fecha civil en Costa Rica.

Límites: auditoría de código y ejecución aislada, sin conexión a producción, envío de correos, migraciones, pruebas de navegador/dispositivo ni transacciones contra un replica set real. No se comprobó la configuración efectiva de secretos, índices desplegados, proveedor de correo o plan comercial en producción. Pasar las suites existentes no invalida los defectos reproducidos.

Este trabajo añade el informe; no corrige código ni modifica los cambios locales anteriores.

2. Catálogo exhaustivo de funcionalidades activas

“100% Funcional” se limita a la función descrita y a la evidencia disponible; no certifica el módulo completo. “Básico” indica una capacidad implementada con límites; “Incompleto” señala una protección o integración defectuosa.

Submódulo / Función	Descripción en código	Estado Backend	Estado Frontend	Madurez
Alta manual	Título, tipo, responsable, prioridad, fechas, animal/potrero y observaciones	Implementada; entrada demasiado amplia	Formulario disponible	Incompleto
Edición y reasignación	`PUT /:id`; marca reasignación manual	Valida responsable solo en campo superior; admite campos internos	Edición y reprogramación	Incompleto
Todas las tareas	Consulta de finca con filtros y búsqueda	Administrador/Encargado; sin paginación de servidor	Tabla paginada localmente	Básico
Mis tareas y detalle	Asignación forzada al usuario en listado; autorización de detalle	Implementados	Vista según rol y apertura desde avisos	Básico
Búsqueda relacionada	Texto, animales, lotes, camadas, ciclos, bandas y campañas	Escapa regex y limita texto a 80 caracteres	Búsqueda con espera de 350 ms	100% Funcional
Estados	Pendiente, En proceso, Completada, Cancelada	Enums; transiciones no uniformes	Botones y formulario	Incompleto
Completar	Guarda fecha, observaciones y efectos derivados	Ruta dedicada con idempotencia opcional	Acción online y cola offline	Incompleto

Submódulo / Función	Descripción en código	Estado Backend	Estado Frontend	Madurez
Comentarios	Usuario y fecha del servidor	Control por rol/asignación; sin notificación	Lista y formulario; recorta espacios	Básico
Eliminación	Borrado físico y limpieza de eventos de Tareas	Sin validación de referencias de origen	Disponible a gestores	Incompleto
Agenda	Próximas actividades de 7 o 30 días	Usa las APIs de tareas	`CalendarioOperativo`: primeras 8 y enlace a Tareas	Básico
Información lunar	Fases, iluminación y próximas fases	No se persiste en Tarea	`astronomy-engine`, zona de organización, opción de ocultar	100% Funcional
Sanidad	Tareas de aplicaciones de planes y tratamientos	Sincroniza desde Sanidad; bloqueo parcial de finalización genérica	Redirige al registro clínico con conexión	Incompleto
Reproducción bovina	Seguimiento de parto, celo y destete	Generación, cancelación y actualización; destete puede escribir Animal	Tareas y detalle	Básico
Reproducción porcina	Celo, desparasitación, alimentación previa y revisión de parto	Generador por fechas del registro	Filtros y tareas porcinas	Básico
Camadas	Hierro, vitaminas, destete, alimentos, venta y otros hitos según destino	Generador y eventos de camada; no ejecuta todos los hechos de negocio	Tareas visibles	Incompleto
IATF	Tareas por paso, ventanas y responsables	Ejecución especializada; edición genérica permite alterar protección	Formulario de actividad, productos e inseminación	Incompleto
Bandas porcinas y engorde	Vinculación de tareas a pasos/ejecuciones	Ejecutores propios; PUT permite completar sin ellos	Formularios especializados	Incompleto
Bancos forrajeros	Siembra, primer corte y siguiente corte	Registrar corte completa pendiente y programa siguiente	Tareas y módulo de bancos	Básico
Tareas de lote	Alta contextual y evento TAREA_PROGRAMADA	Servicio específico	Gestión desde lote	Básico
Aviso de asignación	Aviso operativo al responsable	Falla con `req.usuario`; dedup de toda la vida de tarea/usuario	Campana/centro	Incompleto
Avisos de actividad	Cambios y completados, más emisores en otros módulos	Reparto por rol; actor y finca problemáticos	Campana/centro	Incompleto

Submódulo / Función	Descripción en código	Estado Backend	Estado Frontend	Madurez
Recordatorio interno	Próximas hasta 7 días y vencidas	Cron, dedup por tarea/estado/fecha	Aviso y navegación a tarea	Básico
Contador y lectura	Contar, leer una o todas las propias	Destinatario impuesto desde autenticación	Polling, lectura individual/masiva	Básico
Centro completo	Listado filtrable y paginación API	Hasta 100 por página	Solo pide primeras 100; sin navegación de páginas	Incompleto
Correos operativos	Grupos de alertas próximas/vencidas/críticas	Pro/Premium vigentes, administradores, Resend	No es una preferencia individual del trabajador	Básico
Resúmenes configurables	Diario/semanal, hora, día y módulos	Premium; fincas accesibles y permiso financiero	Formulario en ConfiguracionCuenta, bloqueo Premium y filtro financiero	Básico
Cron	HTTP con Bearer, scripts y programador interno	Bloqueo compartido y registro de ejecución	Sin panel específico auditado	Básico
Lectura offline	Colecciones por contexto y filtros	API suministra datos	IndexedDB v3; requiere carga previa	Básico
Finalización offline	Cola, clave, versión, reintentos y conflictos	Control parcial de concurrencia	Reintentar/descartar; pierde hora de campo	Incompleto

Modelo real y APIs

En Tarea.js los nombres efectivos son **`tipo`**, **`fechaProgramada`**, **`fechaLimite`**, **`fechaCompletada`**; no `tipoTarea` ni `fechaVencimiento`. Contiene `asignadoA`, `creadoPor`, `animal`, `potrero`, `lote`, `moduloOrigen`, `referenciaId`, `creadoAutomaticamente`, `categoriaAutomatica`, `claveAutomatica`, campos de cancelación automática, bitácora, comentarios y `operacionesIdempotentes`. Este último campo está oculto por defecto y la ruta de completar conserva las últimas 50 operaciones. El esquema usa timestamps y concurrencia optimista.

El plugin añade `organizacionId` y `fincaId`. No hay `referenciaModelo`, datos lunares persistidos ni un índice único para la identidad lógica de tareas automáticas. La interpretación de `referenciaId` depende del módulo, categoría y clave.

En Notificacion.js existen **`destinatario`**, **`actor`**, **`naturaleza`**, **`tipo`**, **`titulo`**, **`mensaje`**, **`url`**, **`leida`**, **`fechaLectura`**, entidad, módulo, metadata y dedupKey. Tiene organización, **no finca**. El índice único parcial combina organización, destinatario y dedupKey.

`AlertaCorreo` guarda clave, referencia, fecha objetivo, último envío y cantidad de recordatorios; `ConfiguracionEmail` guarda preferencias y último periodo; `TrabajoProgramado` guarda bloqueo, token, estado, tiempos y resultado.

Base	Métodos y rutas reales
`/api/tareas`	`GET /`, `GET /mis-tareas`, `GET /:id`, `POST /`, `PUT /:id`, `PATCH /:id/estado`, `PATCH /:id/completar`, `POST /:id/comentarios`, `DELETE /:id`
`/api/notificaciones`	`GET /`, `GET /no-leidas/count`, `PATCH /marcar-todas-leidas`, `PATCH /:id/leida`
`/api/cron`	`POST /alertas`, `POST /resumenes-email`

Base	Métodos y rutas reales
`/api/usuario`	`GET /configuracion-emails`, `PUT /configuracion-emails`

Referencias: rutas de tareas, notificaciones, cron, preferencias, montaje en `backend/app.js:30`.

3. Flujo de tareas: creación → asignación → calendario → completado

Creación y asignación

Administrador y Encargado crean tareas. `validarUsuarioAsignable` exige membresía activa en la organización, usuario activo, rol admitido y acceso a la finca actual. Los roles asignables para Tareas son Administrador, Encargado, Trabajador, Veterinario y Contador; Consulta no se ofrece.

La validación es correcta para el campo `asignadoA` normal. No protege operadores anidados ni impide que otros campos trasladen la tarea a otra finca. El controlador fija `creadoPor` en el alta, pero en edición no lo protege con un DTO.

La interfaz inicializa Pendiente, prioridad Media y tipo Otro. Su normalizador de edición copia la tarea completa al formulario y luego al payload; corregir el backend exige también enviar solo campos editables desde React.

Fuentes: `backend/services/usuarioAsignable-service.js:5`, `backend/controllers/tareaController.js:99`, `:264`, `:288`; `frontend/src/pages/Tareas.js:141`, `:330`.

Estados, fechas y comentarios

Los cuatro estados reales son **Pendiente, En proceso, Completada y Cancelada**. “Vencida” es un cálculo visual o de alertas, no un estado guardado. No existe “En progreso”.

La ruta `/completar` rechaza completadas y canceladas y comprueba la versión cuando recibe una fecha válida. `/estado` no valida el estado de origen: un trabajador asignado puede reabrir una completada/cancelada o completar una cancelada. Los gestores pueden editar el estado por PUT.

La fecha de finalización normal es `new Date()` del servidor. PUT también permite proporcionar `fechaCompletada`; no hay una política uniforme de fechas ni validación de `fechaLimite >= fechaProgramada`. Reprogramar desde la interfaz cambia solo `fechaProgramada`, aunque los recordatorios prefieren `fechaLimite`.

Los comentarios se añaden con usuario y fecha del servidor. No generan avisos ni forman parte de la cola offline. El backend solo rechaza un texto falsy: espacios pueden acabar almacenados como texto vacío; no se define una longitud máxima específica.

Sincronización real con otros módulos

Origen	Qué genera	Qué significa completar desde Tareas
Sanidad	Próxima aplicación de plan o tratamiento, con clave por fecha	Debe ejecutarse desde Sanidad. PATCH bloquea; PUT puede eludirlo mediante operadores o alterando previamente la categoría. El registro clínico es quien completa las tareas sanitarias.
Reproducción bovina	Revisión de parto, próximo celo y destete	Marca tarea y crea evento según configuración. Solo el caso de destete tiene un efecto adicional explícito sobre `Animal.fechaDestete`, con comprobación de madre/ciclo. Revisar parto no crea ternero.

Origen	Qué genera	Qué significa completar desde Tareas
Reproducción porcina	Revisión de celo, desparasitación previa, alimento de lactancia y revisión de parto	No crea por sí sola una monta, aplicación clínica, suministro o camada; puede escribir eventos con tipos que parecen esos hechos.
Camada	Hitos sanitarios, alimento, destete, venta/sacrificio y reproducción según destino	Puede crear EventoCamada, pero completar `camada-destete` aquí no ejecuta el destete real de la camada ni sus cantidades.
IATF	Pasos con fecha y ventana, vinculados a ejecución	Formulario propio para productos, participantes e inseminación. El controlador genérico intenta impedir finalización, con los bypass documentados.
Banda porcina / engorde	Tareas de pasos con referencia al protocolo	Los ejecutores actualizan tarea y registros específicos. PUT genérico puede dejar la tarea completada y la ejecución pendiente.
Banco forrajero	Siembra y cortes	Registrar un corte en su módulo completa la tarea pendiente y programa la siguiente. Completarla en Tareas no crea CorteForraje ni programa el siguiente corte.
Lote	Tarea contextual y evento de programación	El controlador genérico no registra automáticamente la finalización en EventoLote ni ejecuta pesajes, sanidad o cambios de ración.

Fuentes: `sanidad-tarea-service.js:40`, `:155`; `reproduccionBovina-service.js:80`; `reproduccionPorcina-service.js:46`; `camada-service.js:240`, `:382`; `iatf-service.js:222`; `protocoloPorcino-service.js:78`, `:293`; `protocoloEngorde-service.js:69`, `:179`; `forraje-service.js:59`, `:111`; `lote-service.js:446`; todos en `backend/services/`.

No se encontró generación directa de tareas en el controlador de rotaciones ni en los servicios generales de alimentación revisados. Las tareas de alimentación observadas proceden de reproducción/camadas y protocolos; no debe asumirse que cualquier suministro o rotación programa una tarea.

Calendario y luna

La agenda usa fecha programada, muestra tareas activas para 7/30 días y presenta hasta ocho actividades. No es una cuadrícula mensual con arrastre. Está integrada en el dashboard; Tareas incorpora sus filtros directamente, sin depender de un archivo `FiltroTareas.js`.

El cálculo lunar es **astronómico y dinámico**, mediante `MoonPhase`, `Illumination`, `SearchMoonQuarter` y `NextMoonQuarter` de `astronomy-engine`. Se consideran la zona horaria de la organización y una fecha representativa; hay opción de ocultar la información. Las fases se muestran en tareas de reproducción y siembra según campos estructurados, no por palabras del título. No se persisten en MongoDB ni ejecutan decisiones clínicas.

Fuentes: CalendarioOperativo.js, calendarioLunarService.mjs, tareasLuna.mjs.

4. Centro de notificaciones y motor de correos

Avisos internos

- Asignación: aviso operativo al responsable, deduplicado por tarea y destinatario.
- Modificación/finalización: avisos informativos según rol del actor. Trabajador/Veterinario → Encargado/Administrador; Encargado/Contador → Administrador; Administrador no genera escalamiento por

defecto.
- Otros módulos incluyen emisores de actividad sanitaria, reproducción, pesajes y finanzas. Que exista la llamada no garantiza entrega: varios pasan el mismo `req.usuario` incompatible descrito en TN-07. No se comprobó un aviso específico de nacimiento entregado de extremo a extremo.
- Cron: avisos al asignado para tareas próximas o vencidas. No valida nuevamente la membresía ni el acceso a finca del responsable; solo comprueba el usuario poblado y su estado.
- No hay aviso de comentario ni de eliminación de tarea.

El servicio expone un punto para un emisor en tiempo real, pero no se encontró su configuración en la aplicación. La campana hace **polling cada 45 segundos**, al enfocar la ventana y al abrirla. Solicita ocho avisos y el contador en paralelo. No es WebSocket, SSE ni push del sistema operativo.

El backend impone el destinatario autenticado para listar, contar y marcar lectura. La campana y el centro pueden mostrar lectura local aunque falle la petición individual; no comparten un mecanismo inmediato para refrescar el contador entre ambos. El centro limita la consulta a 100 y no implementa cambio de página.

La navegación React usa `entidadTipo`, `entidadId` y `moduloOrigen`, no sigue directamente `url`. Una tarea de otra finca no lleva información suficiente para cambiar a su finca antes de abrirla.

Fuentes: `backend/services/notificacion-service.js:12`, `:62`, `:82`, `:123`; `tarea-notificacion-service.js:7`; `alertasCorreo-service.js:59`; `frontend/src/Components/CampanaNotificaciones.js:27`, `:41`; `pages/CentroNotificaciones.js:16`; `Components/ListaUsuario.js:321`.

Cron y contenido de correos

`POST /api/cron/alertas` y `/resumenes-email` exigen `Authorization: Bearer <CRON_SECRET>`. Sin secreto configurado responden 503; con credencial inválida, 401. La comparación usa igualdad de longitud y `crypto.timingSafeEqual`.

El programador interno está activo por defecto (`CRON_MODE=internal`), ejecuta al arrancar y después:

- Alertas: cada 24 horas por defecto, configurable con `EMAIL_ALERTS_INTERVAL_MS`.
- Resúmenes: cada hora por defecto, configurable con `EMAIL_DIGEST_INTERVAL_MS`.

Hay scripts para ejecución externa. El bloqueo MongoDB usa clave única, token y vencimiento de 30 minutos por defecto; una ejecución ocupada se omite. No hay renovación del bloqueo mientras trabaja ni aislamiento de errores por organización.

Los recordatorios operativos no son un resumen individual diario del trabajador: agrupan tareas **próximas, vencidas y críticas** por asunto y envían a administradores activos de la organización. El objetivo es `fechaLimite || fechaProgramada`; se incluyen los próximos siete días y todo lo vencido. `EMAIL_TEST_TO` sustituye los destinatarios si está configurado; su valor real no se consultó.

El envío usa Resend mediante `fetch`. Sin API key devuelve "no enviado" y escribe el mensaje preparado en logs. No se comprobó entrega real. `EMAIL_ALERTS_ENABLED=false` desactiva estos recordatorios, pero no es un interruptor global para los resúmenes Premium.

Deduplicación

Canal	Mecanismo	Límite
Asignación interna	Tarea + usuario, índice único con organización	Volver a asignar al mismo usuario reutiliza el aviso antiguo, incluso leído.

Canal	Mecanismo	Límite
Próxima/vencida interna	Tarea + estado próximo/vencido + fecha objetivo + destinatario	Un aviso de “dentro de 7 días” no se actualiza diariamente. Próxima y vencida son claves distintas.
Correo operativo	AlertaCorreo por tarea/tipo; fecha objetivo y último envío	Próximas cada 6 días; vencidas cada 2; urgentes vencidas cada 1. Son repeticiones intencionadas.
Resumen Premium	Último periodo diario o semanal del usuario	Se marca después de que el proveedor acepte el correo; un fallo posterior puede repetirlo.
Cron global	Bloqueo compartido por trabajo	Reduce concurrencia; no garantiza entrega exactamente una vez ante caída o expiración.

Los correos se registran después del envío. No hay clave idempotente de entrega al proveedor ni registro durable previo de cada entrega. `correos.enviadas` cuenta alertas incluidas, no mensajes físicos: un correo con diez tareas incrementa el contador en diez. Esta diferencia debe aclararse antes de usar la métrica para consumo comercial.

Resúmenes Premium

Permiten desactivar, frecuencia diaria/semanal, hora, día y módulos. El proceso comprueba membresía activa, usuario activo y fincas accesibles; finanzas se restringe por permiso. Agrupa finanzas por moneda y tareas por destinatario. La fecha de envío usa zona horaria de la organización.

La selección global exige organización Activa y plan PREMIUM Activo/Prueba. La ruta de preferencias exige `requireFeature('configuracionEmailAvanzada')`. Son controles más completos que los del reparto general de avisos. La preferencia de resumen ganadero no aplica una comprobación equivalente a `inventario.ver`; conviene definir expresamente si los totales agregados se permiten a Contador.

`frontend/src/pages/ConfiguracionCuenta.js:13` implementa el formulario de frecuencia, hora, día y contenido, muestra la oferta Premium cuando falta el feature y oculta finanzas a roles sin acceso.

Fuentes: `backend/middleware/cronAuth.js`; `services/trabajoProgramado-service.js:11`, `:105`; `services/alertasCorreo-service.js:10`, `:100`, `:171`; `services/correoElectronico-service.js:13`, `:42`; `services/resumenEmail-service.js:117`, `:161`; `services/permisosResumenEmail-service.js:3`.

5. Evaluación de funcionamiento offline (PWA)

IndexedDB usa `ganaderia-romilio-offline`, versión 3. Guarda tareas, inventario, potreros, eventos de animal, cambios pendientes y metadata. Las colecciones se separan por **organización:finca:usuario** y por variante de consulta. La migración desde la primera versión elimina almacenes que no aislaban clientes. La PWA precachea recursos de aplicación; la API usa `cache: 'no-store'`.

No hay descarga completa automática de la finca: se guarda lo consultado. Administrador/Encargado descargan todas las tareas que coincidan con filtros; los demás descargan Mis tareas. Un dispositivo que nunca consultó una combinación de filtros no dispone de ella offline. El rango mensual predeterminado puede dejar fuera tareas vencidas de meses anteriores.

Al completar sin conexión:

1. React guarda una operación `completar-tarea`, observaciones y `versionEsperada=tarea.updatedAt`.
2. IndexedDB añade identificador, clave idempotente, contexto y `creadoEn`.
3. La tarea se muestra localmente como completada y pendiente de sincronizar.
4. Al iniciar la pantalla principal, recuperar conexión o cada 60 segundos, se intenta enviar la cola. También existe acción manual.

5. Se envían observaciones, clave y versión; **no se envía `creadoEn` como fecha real de ejecución**.
6. El servidor usa la hora de recepción y después ejecuta efectos/bitácora.
7. En éxito se elimina la operación; 409 produce Conflicto; otros fallos reintentan con espera creciente y dejan Fallido después de cinco intentos automáticos.

Hay exclusión de sincronizaciones simultáneas dentro de la misma instancia JavaScript, no un bloqueo compartido entre pestañas. El servidor aporta control de versión e idempotencia, con los defectos TN-03/TN-04.

Las tareas clínicas, IATF, bandas y engorde requieren conexión en la interfaz, lo cual evita simular ejecuciones especializadas con un simple cambio de estado.

Limitaciones observadas:

- Hora de campo perdida en tarea, bitácora y posible destete bovino.
- El guardado por filtros crea varias copias de una tarea: completar una no actualiza todas las variantes ni la agenda.
- La lectura desde caché en el `catch` no vuelve a combinar siempre la cola actual.
- Reintentar un conflicto restablece intentos, pero conserva la versión antigua: un cambio real del servidor puede producir 409 indefinidamente hasta descartar/revisar.
- Los datos locales se pueden mostrar también ante un error HTTP de autorización porque el fallback no se limita a fallos de red. Si cambió el rol o acceso, hace falta invalidación explícita del alcance guardado.
- El sincronizador lee el contexto al obtener la cola, mientras la API vuelve a leer la sesión al enviar cada operación. Falta una comprobación de contexto estable durante cambios de sesión/finca; no se reprodujo una carrera de navegador en esta auditoría.

Fuentes: `frontend/src/services/offlineStorage.js:20`, `:148`, `:183`, `:231`, `:266`; `offlineSync.js:22`, `:64`, `:95`; `pages/Tareas.js:186`, `:226`, `:460`; `Components/ListaUsuario.js:249`; `services/api.js:29`; `frontend/vite.config.js`.

6. Matriz de permisos por rol

La tabla describe el camino normal autorizado; los bypass se documentan por separado.

Capacidad	Administrador	Encargado	Trabajador	Veterinario	Contador	Consulta
Ver todas las tareas de la finca	Sí	Sí	No	No	No	No
Ver tareas propias	Sí	Sí	Sí	Sí	Sí	Sí, si existen asignaciones previas
Crear, editar, reasignar, eliminar	Sí	Sí	No	No	No	No
Completar tarea genérica	Cualquiera de la finca	Cualquiera de la finca	Asignada	Asignada	Asignada	No
Cambiar estado genérico	Sí	Sí	Asignada, destinos limitados	Igual	Igual	No
Cancelar por `/estado`	Sí	Sí	No	No	No	No

Capacidad	Administrador	Encargado	Trabajador	Veterinario	Contador	Consulta
Comentar	Cualquiera de la finca	Cualquiera de la finca	Asignada	Asignada	Asignada	No
Ser nuevo responsable desde Tareas	Sí	Sí	Sí	Sí	Sí	No
Leer/marcar sus notificaciones	Sí	Sí	Sí	Sí	Sí	Sí
Configurar resumen Premium propio	Sí	Sí	Sí	Sí	Sí	Sí
Incluir finanzas en resumen	Sí	No	No	No	Sí	No

No pueden asignar normalmente a cualquier miembro de la organización: el selector y la validación exigen acceso a la finca actual y rol permitido. Esto no evita TN-01.

Un peón/Trabajador puede completar una tarea genérica asignada y dejar observaciones/comentarios; no puede crearla ni reasignarla. Ejecutar un protocolo requiere además el permiso y flujo del módulo de origen. Una tarea genérica no debe convertirse en autorización implícita para registrar una operación clínica o económica.

Consulta es solo lectura de tareas; marcar una notificación como leída es una preferencia personal que el backend sí permite. Contador solo ve sus tareas, aunque pueda acceder a finanzas.

Fuentes: `backend/config/permisosRoles.js:9`; `backend/controllers/tareaController.js:68`, `:230`, `:249`, `:341`, `:514`; `backend/services/usuarioAsignable-service.js:5`; `backend/controllers/configuracionEmail-controller.js:40`.

7. Alineación con el modelo SaaS y los cuatro planes

Capacidad	ESENCIAL	GESTIÓN	PRO	PREMIUM	Cumplimiento real
`centroAlertas`	Sí	Sí	Sí	Sí	Configurado en todos; no se exige el feature en las rutas.
Tareas básicas	Disponible	Disponible	Disponible	Disponible	Sin feature exclusivo; permisos por rol.
`emailsOperativos`	No	No	Sí	Sí	Cron comprueba `tieneFeature`, que también verifica vigencia.
`configuracionEmailAvanzada`	No	No	No	Sí	Guard de preferencias y filtro de plan en procesador.

Capacidad	ESENCIAL	GESTIÓN	PRO	PREMIUM	Cumplimiento real
Mutaciones de tareas con suscripción suspendida/cancelada	Deben bloquearse	Deben bloquearse	Deben bloquearse	Deben bloquearse	**No bloqueadas por plan** si organización/membresía siguen activas.
Mutaciones de lectura de notificaciones con plan no vigente	Política por definir/aplicar	Igual	Igual	Igual	Actualmente permitidas; no hay guard de vigencia.

`auth` rechaza organización no Activa, pero solo calcula `req.planVigente`; no lo aplica a estas rutas. No equivale a `requirePlanVigente`.

Los recordatorios internos se crean antes de consultar `emailsOperativos`: una organización Activa con plan Suspendido/Cancelado sigue pudiendo generar avisos internos, aunque no reciba correos operativos. El trabajo general también sincroniza categorías animales, una responsabilidad adicional al envío de avisos.

Precios: Agents.md menciona $10/$20/$35/$60. Los valores predeterminados del código son **$6/$12/$20/$35**, configurables mediante variables `PLAN_PRECIO_*_USD`. No se deduce de esto un error de cobro ni el precio efectivo desplegado; hay una divergencia documental que debe reconciliarse.

Fuentes: `backend/config/planes.js:32`, `:68`, `:101`, `:134`; `services/plan-service.js:174`; `middleware/auth.js:137`; `routes/tareaRoutes.js:16`; `app.js:35`; `services/alertasCorreo-service.js:191`; `services/resumenEmail-service.js:161`.

8. Plan de acción priorizado

Hallazgos P1

TN-01 — Entrada de tareas permite escritura entre organizaciones/fincas y relaciones no autorizadas

Evidencia: `tareaController.js:99`, `:272`, `:311`; `models/plugins/organizacion-plugin.js:67`, `:114`; `services/lote-service.js:449`.

`limpiarDatosTarea` conserva casi todo el cuerpo. El plugin filtra las consultas pero solo asigna organización/finca al validar si faltan; no las hace inmutables. En actualizaciones ordinarias tampoco elimina `$set.organizacionId` o `$set.fincaId`.

Reproducción: un gestor envía un alta con tenant/finca ajenos o edita una tarea propia con `$set` que los cambia. La sonda con hooks reales de Mongoose confirmó que el filtro conserva el contexto actual y la escritura conserva el destino ajeno. `$set.asignadoA` evita además la validación del responsable. Animal, potrero, lote y referencia no se validan como relaciones de la finca.

Impacto: contaminación o traslado de datos fuera del alcance autorizado, asignaciones inaccesibles y referencias inconsistentes. No implica que un GET ordinario pueda leer cualquier tenant.

Acción/aceptación: DTO explícito por operación, rechazo de operadores y campos internos, contexto impuesto en servidor, relaciones comprobadas en finca y defensas del esquema para cambios de tenant. Cubrir alta, PUT y alta desde lote con tenants A/B y usuarios de fincas distintas.

TN-02 — Finalización de tareas protegidas eludible por edición

Evidencia: `tareaController.js:298`, `:354`, `:406`.

PUT solo bloquea `req.body.estado === 'Completada'` para Sanidad clínica e IATF; omite bandas/engorde. `$set.estado` elude la condición. También se pueden cambiar previamente `categoriaAutomatica` o `moduloOrigen`.

Reproducción confirmada: PUT completa BANDA_PORCINA y PROTOCOLO_ENGORDE directamente; `$set` completa un Plan sanitario. No intervienen sus ejecutores.

Impacto: agenda completada con aplicación, inventario, diagnóstico o ejecución todavía pendiente.

Acción/aceptación: proteger la identidad de origen y centralizar transiciones; todos los métodos deben exigir el ejecutor correspondiente. Probar PUT/PATCH, operadores, dos peticiones consecutivas y campos de origen manipulados.

TN-03 — Replay idempotente anterior a la autorización

Evidencia: `tareaController.js:418`, `:476`.

La clave se compara antes de comprobar Consulta o asignación; no se vincula la coincidencia al usuario ni al tipo de operación.

Reproducción confirmada: quien completó una tarea y conoce su clave recibe la tarea poblada con HTTP 200 al repetirla después de que se reasigne a otro usuario. El GET normal del mismo usuario sí devuelve 403.

Impacto: divulgación del estado y datos actuales tras perder acceso. No requiere adivinar una clave aleatoria si el usuario conserva la de su propia operación anterior.

Acción/aceptación: autorizar antes de cualquier replay o respuesta de conflicto; vincular operación a actor, acción y solicitud, y validar su resultado según ciclo de vida. Repetir pruebas tras reasignación y cambio de rol.

TN-04 — Guardado parcial e idempotencia que confirma efectos incompletos

Evidencia: `tareaController.js:163`, `:311`, `:469`.

Se guarda la tarea y su clave antes de modificar Animal, borrar/recrear eventos y notificar. No existe transacción ni una operación durable que represente los efectos pendientes. La bitácora se borra antes de reconstruirla.

Reproducción confirmada: fallo simulado de bitácora después de guardar → HTTP 400 con tarea ya completada; repetir la clave → HTTP 200 sin reintentar la bitácora.

PUT tampoco usa condición de versión ni incrementa explícitamente `__v`; `optimisticConcurrency` en el esquema no convierte `findByIdAndUpdate` en una operación con control de versión.

Acción/aceptación: transacción para efectos locales y registro de operación; cola durable de notificaciones tras confirmar; control de versión también en PUT. Inyectar fallos después de cada escritura y probar concurrencia real en replica set, verificando un único resultado consistente.

TN-05 — Eventos de negocio inferidos a partir de una tarea completada

Evidencia: `tareaController.js:124`, `:145`, `:161`; `config/reproduccionBovinaConfig.js:6`; `services/reproduccionPorcina-service.js:52`; `services/camada-service.js:255`.

Revisar parto/celo puede generar eventos Parto/Monta. Tareas Pesaje, Sanidad, Venta o Sacrificio tienen mapeos por tipo aunque no exista un registro de negocio. `generaBitacora:false` no impide el evento cuando hay un tipo deducible.

Reproducción confirmada: completar una tarea Pesaje con `generaBitacora:false` solicita un evento Pesaje sin registrar un peso.

Impacto: la bitácora puede expresar un nacimiento, servicio o intervención que solo fue un recordatorio marcado. En camadas tampoco asegura destete/venta reales.

Acción/aceptación: distinguir "tarea realizada/observación" del hecho clínico, productivo o comercial; los eventos específicos deben proceder de su registro real. Probar que revisar no equivale a parto y que cerrar una tarea no inventa pesajes o aplicaciones.

TN-06 — Borrado y reapertura rompen trazabilidad de origen

Evidencia: `tareaController.js:29`, `:163`, `:491`; referencias de ejecución en `services/iatf-service.js:222`, `services/protocoloPorcino-service.js:82`, `services/protocoloEngorde-service.js:73`.

DELETE elimina físicamente cualquier tarea gestionable y sus eventos de Tareas sin revisar enlaces de protocolos, eventos de lote o avisos. Reabrir elimina la bitácora derivada, pero no revierte `Animal.fechaDestete`. Las notificaciones pueden quedar apuntando a una tarea inexistente.

Impacto: ejecución con referencia huérfana, pérdida de evidencia y desacuerdo entre animal y agenda.

Acción/aceptación: política de cancelación/archivo para tareas de sistema; impedir borrado de referencias activas y registrar compensaciones explícitas cuando correspondan. Verificar protocolos, bitácora, destete y navegación tras cancelar, reabrir y eliminar.

TN-07 — Avisos manuales fallan por identificador de actor

Evidencia: `middleware/auth.js:121`; `tareaController.js:280`, `:325`, `:472`; `services/tarea-notificacion-service.js:7`; `services/notificacion-service.js:12`, `:67`.

`req.usuario` tiene `id`. Los normalizadores leen `_id` o convierten el objeto a texto, obteniendo `[object Object]`. La asignación falla al validar actor; los avisos informativos pueden fallar al consultar membresías con ese valor. Las envolturas seguras solo escriben el error en consola.

Reproducción confirmada: actor `{id,rol}` produce error de cast; el mismo actor como cadena válida sí permite construir la notificación.

Acción/aceptación: normalizador común que acepte únicamente identificadores válidos y soporte `id`; contratos de emisor coherentes; registro durable/reintento y observabilidad de fallos. Probar alta, reasignación y completado con la forma exacta de usuario emitida por auth.

TN-08 — Destinatarios de avisos y correos sin alcance de finca

Evidencia: `models/Notificacion.js:26`; `services/notificacion-service.js:62`; `services/correoElectronico-service.js:13`; `services/alertasCorreo-service.js:59`, `:208`.

`notificarPorRoles` selecciona todas las membresías activas del rol en la organización, sin filtrar sus fincas. Notificacion no registra finca. El cron reúne tareas de toda la organización y envía el grupo a sus administradores, aunque una membresía tenga acceso limitado. El responsable de un aviso automático tampoco se revalida contra membresía/finca.

Reproducción aislada: el reparto por rol consulta sin restricción de finca y el modelo carece del campo. La entrega efectiva de mensajes manuales puede estar enmascarada por TN-07.

Impacto: títulos, actividad o datos de fincas no autorizadas; avisos que no pueden abrirse desde la finca activa.

Acción/aceptación: persistir origen de finca; validar acceso por destinatario y agrupar correos por alcance autorizado. Mantener avisos globales solo cuando estén definidos como tales. Probar dos encargados y dos administradores con

fincas disjuntas, incluida revocación posterior de acceso.

TN-09 — Mutaciones sin validación de suscripción vigente

Evidencia: `routes/tareaRoutes.js:16`; `app.js:35`; `middleware/auth.js:137`; `services/alertasCorreo-service.js:191`.

Auth comprueba organización activa pero no bloquea por plan. Las rutas no llaman `requirePlanVigente` ni `requireFeature('centroAlertas')`. El cron crea avisos antes de comprobar el feature de correo.

Impacto: usuarios con organización activa y plan suspendido/cancelado pueden seguir modificando tareas; continúan avisos internos.

Acción/aceptación: aplicar política de vigencia centralizada a mutaciones y generadores. Definir qué lecturas y preferencias personales se conservan. Probar Activo, Prueba, Suspendido y Cancelado por HTTP, no solo la utilidad de planes.

TN-10 — Transiciones contradictorias entre `/estado` y `/completar`

Evidencia: `tareaController.js:341`, `:365`, `:431`.

Un asignado no gestor solo tiene una lista de estados destino permitidos; no se comprueba desde qué estado viene. `/completar` rechaza Cancelada, mientras `/estado` permite pasarla a Completada. Tampoco protege la reapertura de una tarea cancelada por el origen.

Reproducción confirmada: trabajador asignado completa Cancelada por `/estado` con HTTP 200.

Acción/aceptación: máquina de estados compartida, permiso explícito de reapertura y restricciones según origen. Probar todas las parejas de estados y rutas; repetir completar no debe cambiar su fecha histórica.

Hallazgos P2

TN-11 — Se pierde la fecha real de ejecución offline

Evidencia: `frontend/src/pages/Tareas.js:460`; `services/offlineSync.js:32`; `backend/controllers/tareaController.js:452`.

La cola conserva `creadoEn`, pero el envío no lo utiliza; backend siempre escribe ahora. La sonda confirmó que incluso una fecha aportada al endpoint de completar se ignora. Se desplazan tarea, evento y eventualmente destete.

Acción/aceptación: transmitir `fechaEjecucionReal` validada y guardar separadamente recepción/sincronización y procedencia offline. Simular varias horas sin señal y comprobar ambas fechas y efectos.

TN-12 — Fechas civiles, vencimientos y reprogramación no coinciden

Evidencia: `frontend/src/pages/Tareas.js:87`, `:104`, `:130`, `:383`; `backend/services/alertasCorreo-service.js:16`, `:37`; `backend/controllers/tareaController.js:83`.

El formulario envía `YYYY-MM-DD`, interpretado como medianoche UTC. Mostrarlo con zona local puede retroceder al día anterior en Costa Rica; la sonda lo confirmó. El servicio lunar conserva el día por otra lógica.

La interfaz cuenta vencidas solo si existe fecha límite; el cron usa límite o programada; el texto relativo usa programada incluso si hay otro límite. Reprogramar solo la programada puede mantener avisos en el límite anterior. Los filtros y el cron no usan de forma uniforme la zona de la organización.

Acción/aceptación: distinguir fecha civil de instante, función compartida de objetivo/vencimiento y regla de reprogramación. Probar medianoche UTC/Costa Rica, sin límite, con límite diferente y actividad de protocolo con hora.

TN-13 — Deduplicación interna impide nuevos avisos legítimos y conserva mensajes antiguos

Evidencia: `services/tarea-notificacion-service.js:24`; `services/notificacion-service.js:27`; `services/alertasCorreo-service.js:86`.

Una reasignación A→B→A reutiliza la primera notificación de A, incluso leída. La próxima alerta por fecha mantiene el texto inicial "dentro de N días" hasta que cambie a vencida o se reprograme.

Acción/aceptación: identidad por evento de asignación/revisión y estrategia explícita para actualizar recordatorios. Probar reasignación, reapertura y lectura previa sin duplicar el mismo evento por un reintento.

TN-14 — Cron sin aislamiento de errores ni recuperación durable de entregas

Evidencia: `services/alertasCorreo-service.js:177`, `:212`; `services/resumenEmail-service.js:143`, `:169`; `services/trabajoProgramado-service.js:11`; `services/correoElectronico-service.js:78`.

Un error de una organización aborta el recorrido y puede dejar sin procesar las siguientes. El correo se envía antes de guardar su deduplicación; una caída entre ambos permite repetición. El lock puede vencer durante un proceso largo y no se renueva; `fetch` no tiene timeout explícito.

Acción/aceptación: capturar resultado/error por organización y destinatario, timeout, renovación del bloqueo y cola durable de entregas con identidad estable. Simular proveedor lento, fallo después de aceptación y una organización defectuosa seguida de otra sana. Separar métricas de correos físicos y tareas recordadas.

TN-15 — Contenido editable interpolado sin escape en HTML de correo

Evidencia: `services/alertasCorreo-service.js:161`; `services/resumenEmail-service.js:98`.

El título de tarea/nombre se incluye directamente en HTML. La sonda confirmó que una etiqueta de enlace en el título permanece como HTML. Esto permite modificar el contenido visual o insertar enlaces; no se afirma ejecución de JavaScript en el cliente de correo.

Acción/aceptación: escapar datos de texto mediante plantilla segura y conservar alternativa text. Probar `<`, `>`, comillas, enlaces y caracteres acentuados.

TN-16 — Centro de avisos sin paginación visible y lectura inconsistente

Evidencia: `frontend/src/pages/CentroNotificaciones.js:16`, `:33`; `Components/CampanaNotificaciones.js:65`; `backend/services/notificacion-service.js:123`.

La API pagina pero la pantalla pide solo las primeras 100. No hay forma de avanzar a avisos más antiguos. La lectura individual ignora errores y la campana puede descontar una lectura que el servidor rechazó; el centro no siempre refleja el nuevo estado hasta recargar.

Acción/aceptación: controles de página/cursor, actualización compartida de contador y recuperación visual ante fallo. Verificar 101+ avisos y error de red al marcar lectura.

TN-17 — Caché offline y conflictos carecen de reconciliación completa

Evidencia: `frontend/src/services/offlineStorage.js:148`, `:231`; `pages/Tareas.js:226`, `:471`; `services/offlineSync.js:95`; `Components/CalendarioOperativo.js:63`.

Las variantes por filtros guardan copias independientes; completar no actualiza todas. Un conflicto reintentado conserva la versión antigua. Se reutiliza caché también ante errores de autorización, sin distinguir si cambió el alcance permitido.

Acción/aceptación: datos normalizados por tarea con vistas derivadas, combinación consistente con cola, invalidación ante cambio de autorización y resolución explícita del conflicto con la versión actual. No sobrescribir automáticamente cambios ajenos. Probar agenda/lista, dos filtros, cambio de rol y retorno de conexión.

TN-18 — Cobertura desigual de avisos en tareas automáticas

Evidencia: `services/forraje-service.js:59`; `services/iatf-service.js:222`; `services/protocoloPorcino-service.js:78`; `services/protocoloEngorde-service.js:69`; `tareaController.js:514`.

Estos generadores guardan tareas sin llamar al aviso de asignación, mientras reproducción simple/camadas sí lo hacen y Sanidad lo condiciona a tarea nueva/opciones. No existe un hook general de Tarea que garantice el aviso. Tampoco hay aviso de comentario. El usuario puede descubrir una tarea solo al consultar la agenda o cuando entre en la ventana de recordatorio.

Acción/aceptación: definir qué eventos requieren aviso y emitirlos de forma uniforme después de confirmar la operación de origen, con deduplicación y reintentos. Probar cada generador y comentario según la política acordada.

Orden recomendado y criterios de cierre

1. **Aislamiento y autorización:** TN-01, TN-02, TN-03 y TN-09. Cerrar la escritura entre alcances, los bypass y el acceso por replay.
2. **Integridad de ejecución:** TN-04, TN-05, TN-06 y TN-10. Unificar transiciones, efectos, bitácora y operaciones de origen.
3. **Entrega segura de avisos:** TN-07 y TN-08 antes de ampliar emisores; después TN-13, TN-14, TN-15 y TN-18.
4. **Trazabilidad y uso en campo:** TN-11, TN-12, TN-16 y TN-17, con pruebas de navegador y reconexión.

Añadir pruebas de regresión de los casos reproducidos, pruebas HTTP con autenticación real y dos tenants/fincas, y pruebas de integración transaccional y de concurrencia. Verificar correos con un proveedor simulado y destinatarios de fincas disjuntas. La auditoría queda documentada; **la remediación de estos hallazgos queda pendiente**.